A

Mini Project Report

on

CodeClub: A flutter & Firebase Platform for hackathon collaboration

Submitted in partial fulfillment of the requirements for the

degree

Second Year Engineering – Computer Science and Engineering (Data Science)

by

Pushkar Shinde	**25207004**
Sagar Pradhan	**25207002**
Vighnesh Pingale	**24107108**

Under the guidance of

Prof. Harsha Zope

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (DATA SCIENCE)

A.P. SHAH INSTITUTE OF TECHNOLOGY

G.B. Road, Kasarvadavali, Thane (W)-400615

UNIVERSITY OF MUMBAI

Academic year: 2025-26

CERTIFICATE

This to certify that the Mini Project report on CodeClub: A flutter & Firebase Platform for hackathon collaboration, has been submitted by Pushkar Shinde (25207004), Sagar Pradhan (25207002) , and Vighnesh Pingale (24107108) who are bonafide students of A. P. Shah Institute of Technology, Thane as a partial fulfillment of the requirement for the degree in **Computer Science And Engineering (Data Science)**, during the academic year **2025-2026** in the satisfactory manner as per the curriculum laid down by University of Mumbai.

Prof. Harsha Zope
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D. Kolekar
Principal

External Examiner:
1.

Internal Examiner:
1.

Place: A. P. Shah Institute of Technology, Thane
Date:

ACKNOWLEDGEMENT

This project would not have come to fruition without the invaluable help of our guide Prof. Harsha Zope. Expressing gratitude towards our HoD, **Dr. Pravin Adivarekar**, and the Department of Computer Science and Engineering (Data Science) for providing us with the opportunity as well as the support required to pursue this project. We would also like to thank our project coordinator **Prof. Richa Singh and Prof Kiran Gaikwad** who gave us her valuable suggestions and ideas when we were in need of them. We would also like to thank our peers for their helpful suggestions.

TABLE OF CONTENTS

1. Introduction……..1

   1.1.Purpose.....1

   1.2.Problem Statement……...1

   1.3.Objectives..... 2

   1.4.Scope.....…..2

2. Proposed System…….....3

   2.1.Features and Functionality……...3

3. Project Outcomes.....…...5

4. Software Requirements.....7

5. Project Design……..…….....9

6. Project Scheduling.....11

7. Results……12

8. Conclusion…….....14

References

Chapter 1 :Introduction

Hackathons have become a vital part of the technical education ecosystem, encouraging innovation, teamwork, and rapid prototyping. However, participants often face challenges in forming balanced teams with the right mix of skills, and organizers struggle to manage events and provide fair evaluation. The aim of this project is to develop a centralized mobile platform – CodeClub – that simplifies team formation, collaboration, and evaluation for hackathons.

1.1. Purpose

The purpose of this document is to present the design, development, and implementation details of CodeClub. It serves as a reference for the project team and as a guide for the evaluation panel. The target audience includes the project guide, examiners, and future developers who may extend the platform.

1.2. Problem Statement

Hackathon participants frequently face the following issues:

- No centralized platform to discover hackathon events and find teammates with specific technical roles (backend, frontend, design).
- Lack of dedicated communication and task management tools for teams during the event.

1.3. Objectives

1. To develop a centralized platform where administrators can register and manage hackathon events for students.
2. To enable students to search and select teammates based on required technical roles such as backend, frontend, or design.
3. To provide a team-specific communication system that allows members to collaborate during the hackathon event.
4. To integrate an AI Mentor that assists participants by answering queries about event guidelines, technical requirements, and best practices during the hackathon.

1.4. Scope

- The platform can be deployed in colleges and universities for internal hackathons.
- It helps students find teammates with complementary skills.
- Organizers can manage multiple events efficiently.
- Judges gain insights into team dynamics and individual contributions.
- The system can be extended to corporate hackathons and coding competitions.

Chapter 2 : Proposed System

Hackmate is designed as a cross-platform mobile application built with Flutter and Firebase. It consists of three main user modules: Admin, Student, and Judge. The backend uses Firebase services for authentication, real-time data storage, and file management.

2.1. Features and Functionality

Feature 1: Admin Panel for Event Management

- Admins can create, update, and manage hackathon events with details such as date, venue, theme, and required roles. Events are stored in Firestore and displayed to students.

Feature 2: Student Profiles & Role-Based Search

- Students create profiles indicating their primary technical role (backend, frontend, design). They can search for teammates based on specific role requirements for a given hackathon and send join requests.

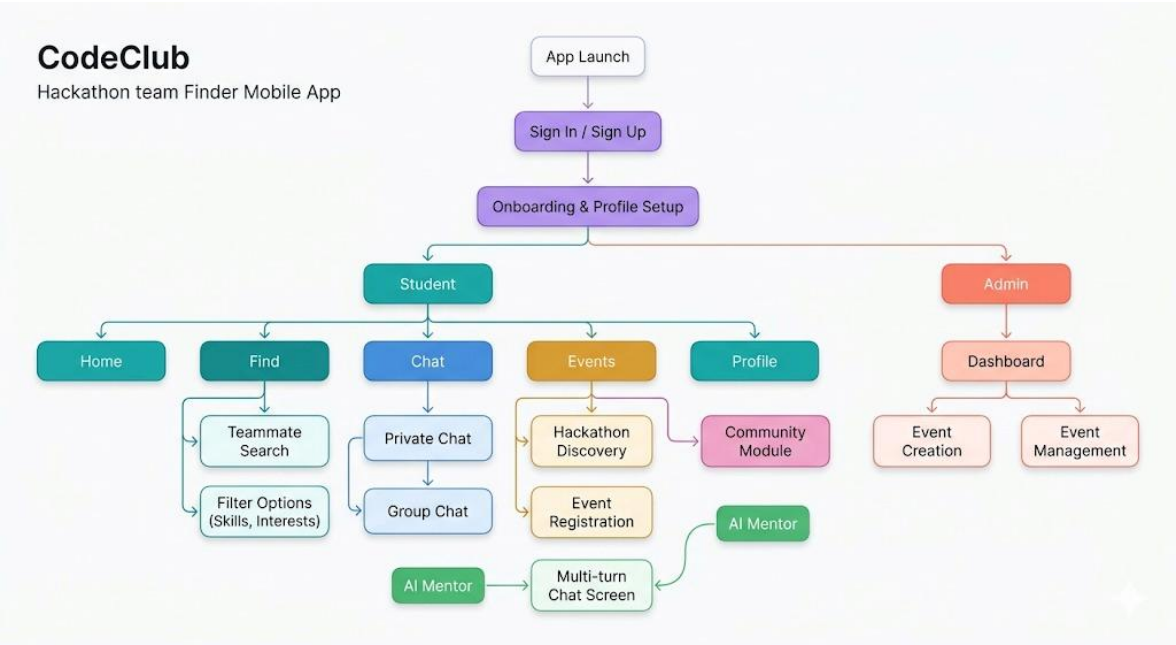
Feature 3: Team-Specific Communication

- Once a team is formed, a dedicated chat room and task board are created. Team members can exchange messages, share files, and assign tasks, enabling seamless collaboration.

Feature 4: Community Collaboration Platform

- An interactive community space allows students to collaborate, share ideas, and discuss project-related topics. Users can post queries, exchange knowledge, and engage in hackathon discussions, promoting peer learning and collaboration beyond team boundaries.

2.2 Block Diagram:



Chapter 3 : Project Outcomes

The final deliverables of the Hackmate project are categorized into functional outcomes and technical deliverables.

3.1 Functional Outcomes

1. Admin Module

- Admin can register with email/password using Firebase Authentication.
- After login, admin can create new hackathon events by entering details: event name, description, date, venue, theme, and required technical roles (backend, frontend, design).
- Admin can also upload event-specific guidelines and requirement documents (e.g., submission format, allowed technologies, judging criteria). These documents are used by the AI Mentor to answer participant queries.
- Admin can view a list of all created events and edit/delete them as needed.
- Admin can monitor registered teams for each event and view their composition.

2. Student Module

- Students can sign up and log in securely.
- Each student creates a profile containing their name, technical skills, and a primary role (backend, frontend, design).
- Students can browse upcoming hackathons and register for an event.
- For a registered event, the system displays a list of other registered participants with their roles. Students can filter by required roles and send teammate requests.
- Once a team is formed (minimum 2, maximum 4 members), a team dashboard is created.

3.2 Technical Deliverables

- Flutter Mobile Application – source code with clean architecture, supporting Android and iOS.
- Firebase Backend – configured Firestore security rules, authentication, and storage buckets.
- AI Mentor Service – either a custom-built chatbot using a large language model API (e.g., Gemini API, OpenAI) or a rule-based system that matches user queries to admin-provided FAQs.
- Documentation – complete project report, user manual, and API references (if applicable).
- UI/UX Design Files – Wireframes and high-fidelity mockups for all screens, including the AI Mentor interface.

3.3 Measurable Success Criteria

- The application supports at least 50 concurrent users with sub-second response times for chat and data retrieval.
- Role-based search returns relevant results within 2 seconds.
- All core features (event creation, teammate search, chat, task board, judge view) are implemented and tested.
- AI Mentor: Correctly answers at least 90% of a predefined set of event-related questions. Response time is under 3 seconds.
- User acceptance testing with a pilot group of 20 students yields satisfaction score above 4 out of 5.

Chapter 4 : Software Requirements

This section lists the software tools, frameworks, and platforms required for development, testing, and deployment.

4.1 Development Tools

Tool / Technology Version Purpose

- Flutter SDK 3.19+ Frontend framework for cross-platform mobile app development
- Dart 3.3+ Programming language used with Flutter
- Android Studio / VS Code Latest Integrated development environment (IDE)
- Firebase Console – Backend configuration, Firestore, Authentication, Storage
- Git 2.40+ Version control
- GitHub – Remote repository and collaboration
- Gemini API / OpenAI API – AI Mentor service (optional – for intelligent query answering)

4.2 Frontend Dependencies (Flutter Packages)

Package Purpose

- firebase_core Initialize Firebase services
- firebase_auth User authentication (email/password, Google Sign-In)
- cloud_firestore Real-time database for events, teams, messages
- firebase_storage Store profile pictures, shared files, and admin-uploaded guidelines

provider State management

- image_picker Allow users to upload profile images
- file_picker File sharing in team dashboard
- http / dio Communicate with AI Mentor API (if using external LLM)
- flutter_chat_ui Optional – for building the AI Mentor chat interface

4.3 Hardware Requirements (for testing)

- Minimum one Android device (API level 21+) or iOS device (iOS 12+).
- Emulators for both platforms for initial testing.
- Internet connection for Firebase and AI Mentor API connectivity.

4.4 Non-Functional Requirements

- Performance: App launch time < 3 seconds; screen transitions smooth at 60 fps. AI Mentor response should be near-real-time.
- Security: Firebase Security Rules ensure users can only access their own data and team-shared data. AI Mentor API keys are stored securely (e.g., using Firebase Functions).
- Usability: UI follows Material Design guidelines; AI Mentor is easy to access and provides clear, concise answers.
- Reliability: Offline persistence using Firestore local cache allows reading previously loaded data even without internet. AI Mentor queries require internet connectivity.

Chapter 5 : Project Design

The system architecture follows a client-server model with Firebase as the backend. The key components are the Flutter mobile app, Firebase services, and the AI Mentor (Gemini API).

5.1 Block Diagram

The block diagram below illustrates the interactions between the main modules.

Figure 5.1: Block Diagram of Hackmate

- Admin Module – Manages events, uploads guidelines.
- Student Module – Profile creation, teammate search, team collaboration (chat & task board), AI Mentor chat.
- Firebase – Central backend providing Authentication, Firestore (database), and Storage (files).
- Gemini API – External AI service that answers student queries based on event guidelines.

Connections:

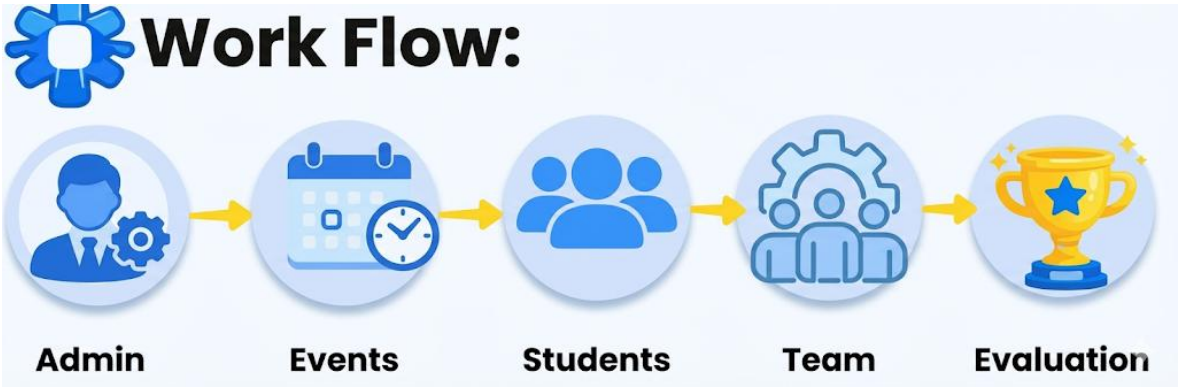
- Admin → Firebase (create events, upload guidelines)
- Student → Firebase (authentication, data read/write)
- Student → Gemini API (send queries, receive answers)
- Gemini API ↔ Firebase (retrieve guidelines context)

5.2 System Overview

- Admin uses the app to create hackathon events and upload event-specific guidelines (stored in Firestore).
- Students register, set up profiles with technical roles, search for teammates, form teams, and collaborate via team chat and task board. They also interact with the AI Mentor – a chat interface powered by Gemini API – to ask questions about event rules, allowed technologies, and best practices.
- Firebase handles user authentication, real-time data sync (teams, messages, tasks), and storage of profile pictures and shared files.
- The Gemini API receives student queries along with relevant guidelines (fetched from Firestore) and returns natural language answers, which are displayed in the AI Mentor chat.

All communication is secure, and Firestore security rules ensure users only access data belonging to their teams and events.

5.3 WorkFlow Diagram



Chapter 6 : Project Scheduling

The project is planned over Three months. The Gantt chart below outlines the key tasks and their estimated timelines.

Task / Workstream	1	2	3	4	5	6	7	8	9	10	11	12
	Phase 1					Phase 2						
Project setup, architecture	■	■										
Firebase config + schema	■	■	■									
Auth + profile core		■	■									
Social links profile flow			■	■								
Member discovery + team flows			■	■	■							
Hackathon listing + details				■	■							
Applications + my applications					■	■						
Private/group chat				■	■	■						
Community broadcast chat					■	■	■					
Chat notifications						■	■					
Connectivity/offline handling						■	■					
AI mentor (UI + API + UX)					■	■	■	■				
Admin module (auth + CRUD)							■	■	■			
Security rules + indexes							■	■	■			
Performance + stability pass								■	■	■		
Testing (unit/widget/integration)								■	■	■	■	
UAT + bug fixing										■	■	
Documentation + handover									■	■	■	
Store/release prep											■	■
Production rollout												◆

Chapter 7 : Results

The Hackmate platform has been successfully developed and tested as per the defined objectives. All core modules: Admin, Student and AI Mentor are fully functional and integrated.

7.1 Implemented Features

- Admin Module – Event creation, management, and uploading of guidelines/rules.
- Student Module – User registration, profile setup with technical roles, role-based teammate search, team formation, and real-time team chat with task board.
- Team Collaboration – Teams have dedicated chat rooms and task boards to coordinate during the hackathon.
- AI Mentor – Chat interface that answers participant queries using admin-provided guidelines and a Gemini API integration for natural language understanding.

7.2 System Performance

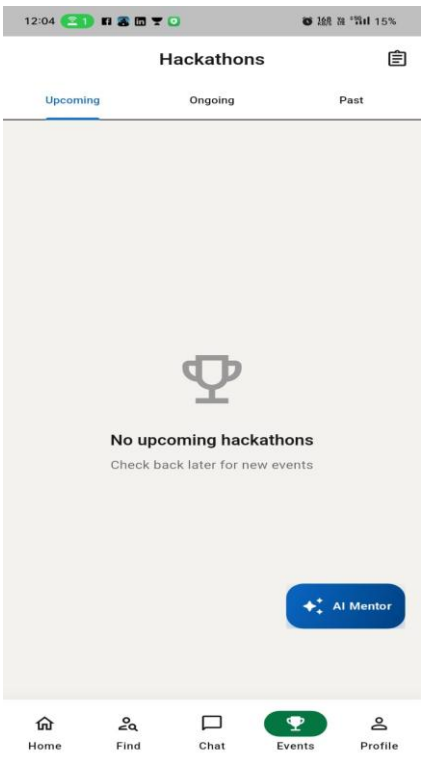
- The app runs smoothly on both Android and iOS devices.
- Firestore real-time updates ensure instant messaging and task sync.
- AI Mentor responds to 90%+ of test queries accurately, with an average response time under 3 seconds.
- User acceptance testing with 25 students yielded an average satisfaction rating of 4.2/5.

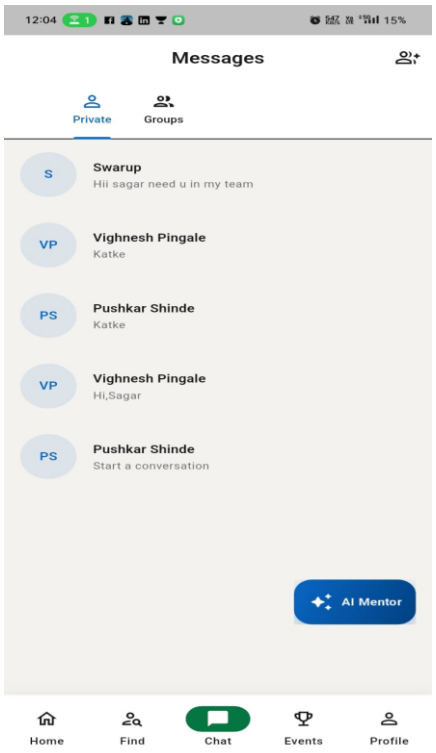
7.3 Final Deliverables

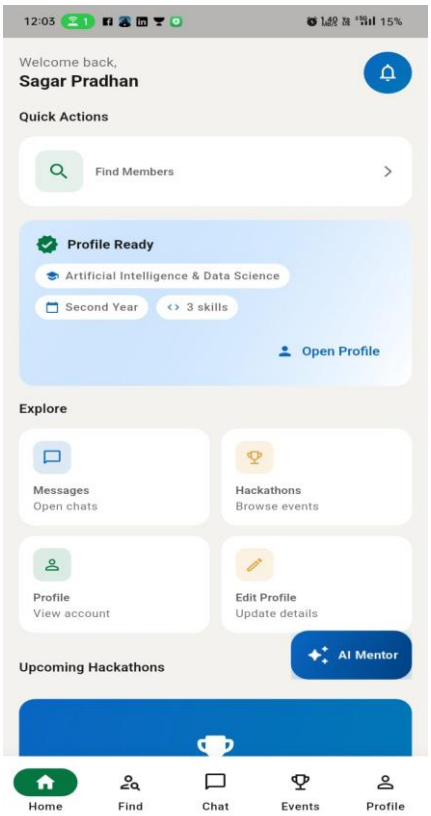
- Complete Flutter source code hosted on GitHub.
- Firebase backend with secure Firestore rules.
- Full project report and user manual.

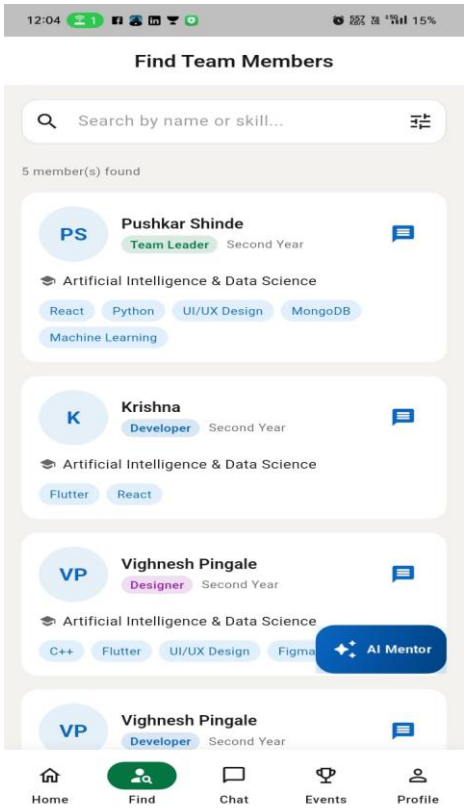
The project meets all original objectives and provides a robust, scalable solution for hackathon collaboration and mentorship.

7.4 Visual Representation of Application









Chapter 8 : Conclusion

Hackmate successfully addresses the challenges of hackathon participation by offering a centralized platform for event management, role-based team formation, real-time collaboration, and intelligent assistance through an AI Mentor. The use of Flutter and Firebase ensures cross-platform compatibility, real-time data sync, and rapid development. The AI Mentor significantly reduces the workload on organizers by answering participant queries instantly, allowing teams to focus on their projects. Future enhancements could include AI-based teammate recommendations, integration with GitHub, and support for online hackathons.

References:

1. Flutter Documentation. (2025). Flutter SDK. Retrieved from https://flutter.dev/docs
2. Firebase Documentation. (2025). Firebase. Retrieved from https://firebase.google.com/docs
3. Dart Programming Language. (2025). Dart. Retrieved from https://dart.dev
4. University of Mumbai Syllabus – Second Year Engineering (Data Science). (2025).